Practical: 5	**Build a small rule-based expert system or logic inference engine for a diagnosis or advisory problem.**

Aim:
To build a small **rule-based expert system** for diagnosing common problems using logical rules.

Theory:

A rule-based expert system uses a collection of IF-THEN rules to make decisions.
Example:
IF fever AND cough
THEN possible flu
Main components:

- Knowledge Base: stores facts and rules.
- Inference Engine: applies rules to available facts.
- User Input: provides symptoms or information.
- Conclusion: gives the appropriate advice.

Algorithm

1. Define rules containing conditions and conclusions.
2. Take symptoms as input from the user.
3. Store the symptoms as facts.
4. Compare the facts with each rule.
5. If all conditions of a rule are satisfied, generate its conclusion.
6. Display the diagnosis/advice.
7. If no rule matches, display that no conclusion was found.

Program:

```
def laptop_expert(turns_on, internet_works, is_slow):

    if turns_on == "no":
        return "Check the charger and power cable."

    elif internet_works == "no":
        return "Check Wi-Fi and restart the router."

    elif is_slow == "yes":
        return "Close unused programs and restart the laptop."

    else:
        return "The system does not find a common problem."


turns_on = input("Does the laptop turn on? yes/no: ").lower()
internet_works = input("Does the internet work? yes/no: ").lower()
is_slow = input("Is the laptop slow? yes/no: ").lower()

advice = laptop_expert(turns_on, internet_works, is_slow)

print("\n--- Expert System Advice ---")
print(advice)
```

OUTPUT

```
Does the laptop turn on? yes/no: no
Does the internet work? yes/no: no
Is the laptop slow? yes/no: yes

--- Expert System Advice ---
Check the charger and power cable.
```

Conclusion

In this practical, a **rule-based expert system** was developed using IF-THEN rules. The system takes user-provided facts and applies logical rules to generate an appropriate advisory result.